devops-flow — Guide d’utilisation

Version 1 · 2026-11-04

Version du 4 novembre 2026 · Application locale · Interface en français

Ce guide décrit l’installation, le diagnostic guidé et la documentation du projet. devops-flow confronte les pratiques déclarées par l’équipe aux configurations du dépôt. Ses résultats ne certifient ni la maturité d’une équipe, ni le fonctionnement d’un pipeline.

1. Préparer l’application

Installation Linux

Depuis le dossier de devops-flow, avec Python 3.11 ou supérieur :

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[pdf]'
python -m playwright install chromium
python -m devops_flow
```

Ouvrir **http://127.0.0.1:8501** dans le navigateur. Arrêter l’application avec `Ctrl+C` dans le terminal. Une connexion Internet est nécessaire pour installer les dépendances et interroger un dépôt distant. L’interface et la base restent locales.

Si un navigateur Chromium ne démarre pas, vérifier ses bibliothèques système : `python -m playwright install-deps chromium`. Cette commande peut demander des droits administrateur. L’export peut utiliser WeasyPrint si ses bibliothèques système sont installées ; le message de l’application indique le moteur réellement utilisé.

Mettre à jour une installation existante

1. Arrêter l’application.
2. Sauvegarder la base utilisée lorsque l’application est arrêtée : `DEVOPS_FLOW_DB_PATH`, sinon `DEVOPS_ASSIST_DB_PATH`, sinon `var/devops_flow.sqlite3` si elle existe, sinon l’ancienne `var/devops_assist.sqlite3`. Aucun fichier n’est déplacé automatiquement.
3. Activer l’environnement existant. Pour une installation DevOps Assist, exécuter `python -m pip uninstall -y devops-assist`, puis installer les dépendances avec les commandes `pip install` et `playwright install` ci-dessus.
4. Relancer l’application. Les nouvelles tables sont créées automatiquement. Les diagnostics existants restent figés.

Le lancement par module devient `python -m devops_flow`. Le dossier du projet peut conserver son nom. La préférence d'apparence antérieure est reprise ; les profils et marqueurs techniques Trello historiques restent compatibles. Les boards existants et les documents utilisateur ne sont pas renommés automatiquement.

Linux et Chromium constituent l'environnement vérifié. Les autres systèmes et navigateurs nécessitent leur propre recette.

2. Créer ou ouvrir un projet

1. Ouvrir l'espace **Projets**.
2. Cliquer sur **Nouveau projet**, puis renseigner le nom et, si utile, une description.
3. Choisir le profil d'évaluation, puis cliquer sur **Créer le projet**.
4. Pour reprendre un projet, utiliser son bouton **Ouvrir**.
5. Cliquer sur **Préparer le diagnostic**.

Dans **Découvrir avec une démo**, le bouton **Charger une démo** crée un projet fictif distinct. Il permet de découvrir les écrans sans fournir de dépôt ni de jeton.

Le bandeau supérieur indique toujours le **Projet ouvert**. **Changer de projet** ramène à la liste ; **Réglages** permet de renommer le projet et de choisir le profil et sa version.

Le sélecteur **Apparence** propose Système, Clair et Sombre. Ce choix reste dans le navigateur ; les questionnaires et jetons ne sont pas enregistrés dans ce stockage.

Un projet utilise une version précise d'un profil. Le profil définit les questions, les contrôles et les seuils du diagnostic. Publier une nouvelle version d'un profil ne modifie pas automatiquement les projets ou leurs diagnostics antérieurs.

3. Préparer et compléter le questionnaire

1. Dans **Diagnostic guidé**, utiliser **1. Préparer** pour choisir pratiques et dépôt, pratiques uniquement ou dépôt uniquement. La carte **Votre prochaine action** indique comment continuer.
2. Ouvrir **2. Questionnaire** et parcourir les cinq aspects : Culture, Automatisation, Amélioration du travail, Mesure et Partage des connaissances (CALMS).
3. Répondre aux questions de votre profil. Choisir **Je ne sais pas** lorsque l'information manque ; cela se distingue de **Jamais**.
4. Ajouter une note facultative si nécessaire.
5. Cliquer sur **Enregistrer le questionnaire**. Le statut confirme l'enregistrement ; il signale toute modification ultérieure non enregistrée.
6. Vérifier les indicateurs et la part des réponses connues, puis continuer vers le dépôt ou les résultats.

Les modifications non enregistrées ne sont pas incluses dans le diagnostic. Un questionnaire enregistré, même entièrement inconnu, peut être utilisé seul. Les scores sont déclaratifs ; aucun

score global de maturité n'est calculé.

4. Choisir une source technique

Dans **3. Dépôt**, choisir une source dans **Analyse du dépôt**. Les étapes restent accessibles librement ; passer d'une étape à l'autre conserve la saisie de la session.

Source	Action
Archive ZIP	Choisir un ZIP de 10 Mio maximum, puis cliquer sur Analyser le dépôt.
Dossier local	Indiquer un dossier sur la machine qui lance l'application, puis analyser.
Dépôt distant	Choisir GitLab/GitHub, saisir l'URL HTTPS et éventuellement la référence et le jeton, puis analyser.

Pour ZIP/local, ces configurations sont indexées dans les sous-dossiers : `Dockerfile`, `compose.yaml`, `compose.yml`, `docker-compose.yaml`, `docker-compose.yml` et `.gitlab-ci.yml`. Les manifests identifient jusqu'à 20 composants. L'onglet **Architecture** présente les stacks Python, Node/TypeScript, Go, Java et Rust, leurs confidences, les fichiers et leurs relations explicites. Les includes et templates ne sont pas résolus ni exécutés. Le mode distant conserve sa portée racine. Aucun code, script, pipeline ou image du dépôt n'est exécuté.

Les ZIP sont inspectés en mémoire : 10 Mio compressés, 1 000 entrées maximum, 20 Mio annoncés au total, 1 Mio par fichier, ratio de compression limité à 100. Les chemins dangereux, liens et entrées spéciales sont refusés. Le dossier local est parcouru avec les mêmes limites de chemins et d'entrées, hors secrets, dépendances et sorties de build. Les points d'entrée candidats sont limités à trois par composant, 200 lignes et 64 Kio chacun.

5. Analyser GitLab ou GitHub

Préparer les accès

Un dépôt public peut être accessible sans jeton, sous réserve des limites du fournisseur. Pour un dépôt privé, créer le jeton depuis votre compte sur le fournisseur, avec une expiration et l'accès limité aux projets nécessaires.

- **GitLab** : `read_api` recommandé pour les fichiers et les métadonnées. Un jeton `read_repository` peut limiter les métadonnées disponibles.
- **GitHub** : jeton finement configuré avec **Contents: Read-only** et **Actions: Read-only** pour l'historique CI. Un jeton classique avec `repo` est également accepté pour un dépôt privé, mais ses permissions sont plus larges.

devops-flow effectue uniquement des lectures. Ne copier le jeton ni dans une URL, ni dans une page documentaire. Il reste en mémoire de session et n'est pas enregistré dans SQLite. Le bouton

Effacer le jeton le retire ; un rechargement complet nécessite une nouvelle saisie. Révoquer les jetons inutilisés depuis le fournisseur.

Lancer l'analyse

1. Choisir **Dépôt distant** et le fournisseur.
2. Saisir une URL de dépôt, par exemple `https://gitlab.com/groupe/projet` ou `https://github.com/proprietaire/depot`.
3. Saisir une branche, un tag ou un SHA si nécessaire. Laisser vide pour la branche par défaut. Ne pas coller une URL de page `/tree/…`.
4. Renseigner le jeton si le dépôt est privé.
5. Cliquer sur **Analyser le dépôt**.
6. Vérifier le SHA acquis, la méthode API/archive, les fichiers ciblés et les constats.

L'application fige la référence en SHA avant lecture. Elle récupère seulement les configurations et manifests ciblés ; si l'API devient indisponible ou trop lente, elle peut essayer une archive, sous les mêmes limites de sécurité. Les instances autohébergées sont acceptées si leur hôte est HTTPS public avec un certificat valide. Les adresses privées, loopback et certificats non valides sont refusés.

Les manifests `pyproject.toml`, `requirements.txt`, `package.json`, `go.mod`, `.python-version` et `.nvmrc` apportent des indices de stack déclarée. Ils ne sont ni exécutés, ni conservés intégralement. Leur présence n'ajoute pas de règle au diagnostic.

Un dépôt GitHub utilise les six contrôles existants. GitHub Actions peut apparaître dans l'historique distant, mais ses fichiers ne font pas l'objet de nouveaux contrôles.

6. Créer et lire le diagnostic

1. Dans **4. Résultats**, vérifier les sources à inclure : questionnaire enregistré, analyse du dépôt de la session, ou les deux.
2. Cliquer sur **Créer le diagnostic**. Si une source sélectionnée manque, le bouton est désactivé et un message indique comment continuer.
3. Lire les scores, les constats, leurs preuves et leurs limites.
4. Utiliser les filtres de statut et de source pour préciser la lecture.
5. Dans le flux DevOps, choisir une étape pour consulter les sources associées.

Au moins une source admissible est nécessaire. Un scan rejeté n'est pas admissible. Une configuration **non observée** n'est pas une preuve d'absence ; **à confirmer** signifie que les informations disponibles ne suffisent pas.

Le diagnostic sauvegardé est immuable. Une nouvelle analyse ne remplace pas l'ancien résultat. Le bouton **Créer un autre diagnostic avec ces sources** permet d'enregistrer volontairement un autre diagnostic.

7. Suivre les actions et l’historique

1. Ouvrir **Plan d’action** et choisir le diagnostic concerné.
2. Modifier le statut des actions : À faire, En cours, Terminée ou Écartée.
3. Ajouter un commentaire et ajuster l’ordre si utile.
4. Ouvrir **Historique / Rapports** pour consulter un résultat antérieur.
5. Dans **Dernières informations du dépôt distant**, consulter l’acquisition, les commits et les exécutions CI disponibles.

L’historique présente au maximum dix commits et cinq exécutions pour le SHA analysé. Une durée indisponible reste inconnue. L’indisponibilité de la CI ne prouve pas son absence et ne transforme pas le diagnostic statique. L’actualisation reste manuelle : relancer **Analyser le dépôt**, puis créer un nouveau diagnostic si souhaité. Aucun job périodique ni webhook n’est activé.

Pour consulter d’autres lectures du dépôt, ouvrir **Consulter toutes les lectures du dépôt**.

Les rapports de diagnostic sont téléchargeables en **Markdown** et **JSON**. Ils excluent les contenus bruts, jetons, chemins absolus et métadonnées distantes libres. Les rapports report-v4 incluent les chemins relatifs validés et l’architecture figée.

8. Créer la documentation du projet

1. Revenir dans **Projets**, ouvrir le projet et cliquer sur **Documentation du projet**.
2. Cliquer sur **Créer une page**.
3. Renseigner le titre et le contenu Markdown.
4. Ouvrir **Aperçu de la saisie** et cliquer sur **Actualiser l’aperçu**.
5. Cliquer sur **Enregistrer la version**.

L’adresse de la page est `/projects/{id}/docs`. Elle reste consultable sans exporter. Les pages restent dans la base locale ; elles ne sont jamais écrites dans le dépôt distant.

Pour ajouter un diagramme, utiliser un bloc clôturé avec le langage `mermaid` :

````
~~~mermaid
flowchart LR
  Code-->Build-->Tests-->Deploiement
~~~
````

Les directives Mermaid, liens actifs, images et ressources externes sont refusés. Une erreur de diagramme laisse le texte consultable et propose une reprise. Limites : titre de 120 caractères, 256 Kio par page, 100 pages par projet, 20 diagrammes par page et 16 Kio par diagramme.

9. Modifier ou restaurer une page

1. Sélectionner la page, puis cliquer sur **Modifier la page**.

2. Adapter le contenu et enregistrer : une nouvelle version est créée.
3. Dans **Versions de la page**, choisir une version antérieure pour la consulter.
4. Cliquer sur **Restaurer comme nouvelle version** pour la reprendre sans effacer les versions existantes.

Si une autre fenêtre a modifié la page, la sauvegarde obsolète est refusée. Votre saisie reste affichée : la copier si nécessaire avant d'utiliser **Recharger la dernière version**. Les brouillons non enregistrés sont perdus lors d'un rechargement complet du navigateur.

10. Exporter la documentation en PDF

1. Dans la documentation, sélectionner les **Pages à exporter**.
2. Cliquer sur **Afficher / actualiser l'ordre**.
3. Utiliser **Monter** et **Descendre** pour organiser le document.
4. Cliquer sur **Télécharger le PDF** et attendre la fin du traitement.
5. Vérifier le message indiquant le moteur utilisé et les éventuels avertissements.

Les révisions sélectionnées au lancement sont figées pour cet export. Le PDF n'inclut pas l'intégralité du diagnostic. Sa sélection est limitée à 2 Mio et 100 pages documentaires ; une page documentaire peut occuper plusieurs pages PDF.

Playwright/Chromium rend les diagrammes. Si Chromium est indisponible ou son rendu échoue, WeasyPrint peut produire le PDF : le code Mermaid remplace alors les illustrations, avec une mention explicite. Vérifier le document avant partage.

11. Résoudre les erreurs courantes

Message ou situation	Action
Accès refusé	Vérifier l'expiration, le dépôt autorisé et les permissions du jeton.
Dépôt ou référence inaccessible	Vérifier l'URL, la branche/tag/SHA et l'accès au dépôt privé.
Limite de requêtes atteinte	Attendre avant de relancer ; utiliser un jeton adapté si nécessaire.
Destination refusée	Utiliser un hôte HTTPS public avec un certificat valide.
Limite de taille ou temps dépassée	Réduire le périmètre du dépôt ; ne pas contourner les bornes de sécurité.
Historique CI indisponible	Vérifier les permissions CI ; le scan des fichiers peut rester valable.
Aucun fichier ciblé	Vérifier les noms, les manifests, les exclusions et le périmètre du composant ; le mode distant reste limité à la racine.

Message ou situation	Action
PDF indisponible	Installer l'extra pdf et Chromium, puis vérifier les dépendances système.
Page modifiée ailleurs	Conserver la saisie, recharger la dernière version, puis réappliquer les changements.

12. Sauvegarder vos données

Arrêter l'application puis copier la base SQLite vers votre sauvegarde habituelle. Elle contient projets, diagnostics, suivis, acquisitions et documentation versionnée. Les jetons et les octets de configuration acquis ne sont pas sauvegardés.

La documentation est du texte libre que vous choisissez d'exporter. Vérifier son contenu avant partage et ne pas y coller de secrets. Aucun envoi cloud ou publication automatique n'est réalisé par devops-flow.

13. Comprendre les contrôles et adapter les questions

Comprendre les contrôles, dans Préparer ou Dépôt, explique le périmètre et les contrôles activés. Les nombres ne représentent pas une norme : l'application propose trois contrôles Dockerfile, un Compose et deux GitLab CI, soit six au total. Un profil peut en désactiver ; les filtres des résultats réduisent seulement ce qui est affiché.

Intitulé affiché	Référence dans les détails
Présence d'un Dockerfile	DOC-001
Utilisation explicite du tag latest	DOC-002
Utilisateur déclaré dans l'image finale	DOC-003
Privilèges élevés dans Compose	CMP-001
Présence d'une configuration GitLab CI	CI-001
Étape de tests identifiable dans GitLab CI	CI-002

Chaque constat présente l'observation, les éléments examinés, ses limites et la suite proposée. Les références internes restent dans **Détails techniques**. La vue du flux utilise les mêmes étapes et statuts avec des titres lisibles ; les exports conservent les références nécessaires à la traçabilité.

Les documentations Docker et GitLab expliquent les pratiques visées ; elles ne valident ni cette sélection limitée de contrôles ni le barème. Détecter `latest` ne mesure pas toute la reproductibilité d'une image. Une instruction USER ne prouve pas l'utilisateur effectif en

production. Le barème et le rapport impact / effort restent des conventions internes, sans certification.

1. Ouvrir **Réglages** pour choisir le profil et la **Version utilisée**, puis enregistrer.
2. Pour adapter les questions et contrôles, utiliser **Modifier les questions et contrôles**. Le profil de base étant protégé, créer un profil personnalisé si nécessaire.
3. Renseigner un nom et les questions ; les identifiants sont proposés automatiquement et restent modifiables dans les options avancées.
4. Activer les contrôles utiles. Les seuils et les pondérations sont dans les options avancées ; la part minimale de réponses connues se saisit en pourcentage.
5. Publier, puis revenir dans **Réglages** pour appliquer explicitement le profil et sa version au projet. Publier ne modifie jamais un projet automatiquement.

Un changement de profil avec des réponses non enregistrées demande confirmation. Les questionnaires enregistrés pour les anciennes versions et les diagnostics existants restent conservés.

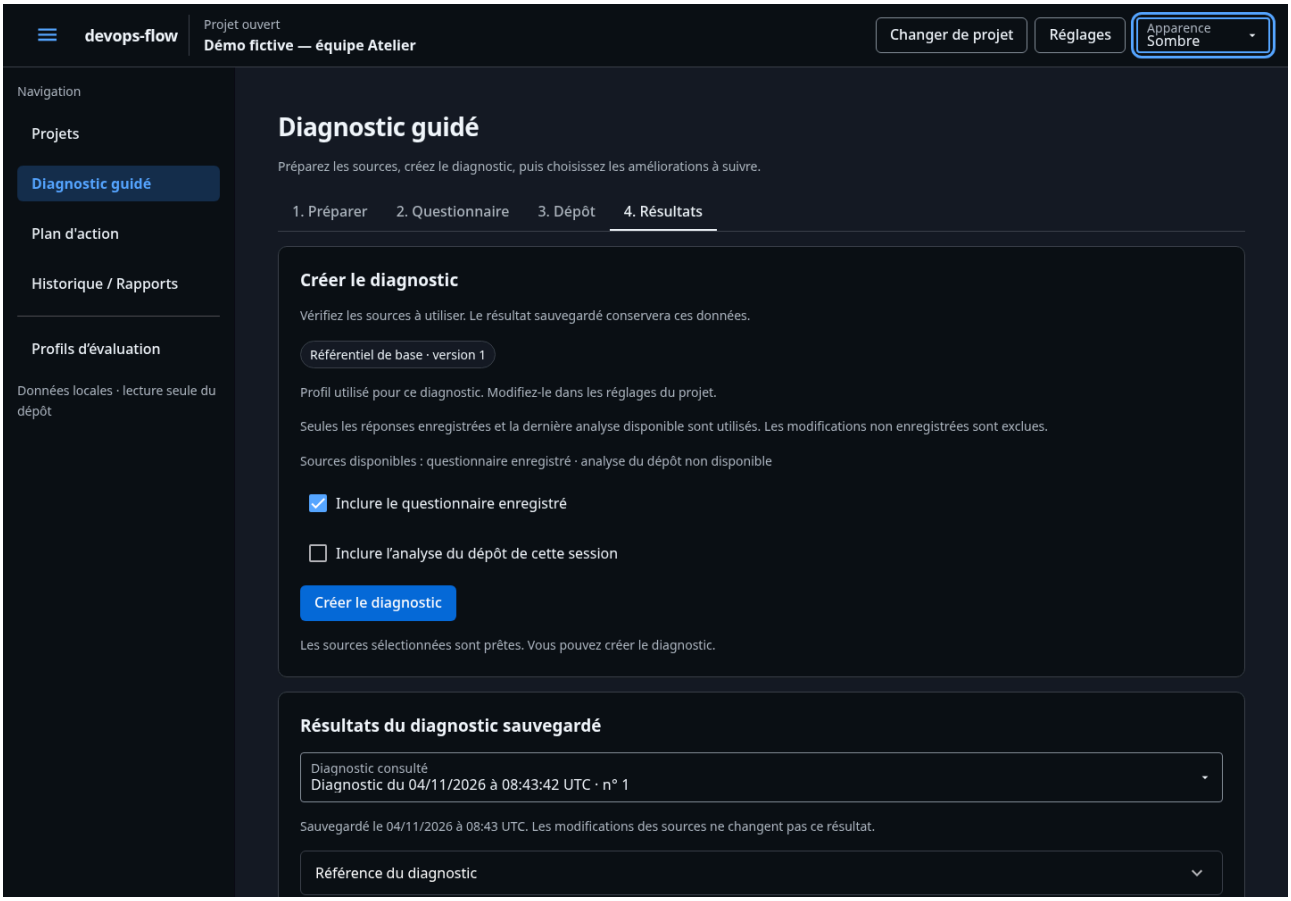
Architecture DevOps et déclarations de stack

Dans **3. Dépôt**, analyser un ZIP ou un dossier local puis ouvrir **Architecture**. Chaque carte présente le chemin du composant, les technologies déclarées et les fichiers DevOps. Ouvrir les signaux pour consulter leur source et leur confiance. Un port repéré dans un point d'entrée est une hypothèse faible ; un framework dans un manifest reste une dépendance déclarée, sans preuve d'usage.

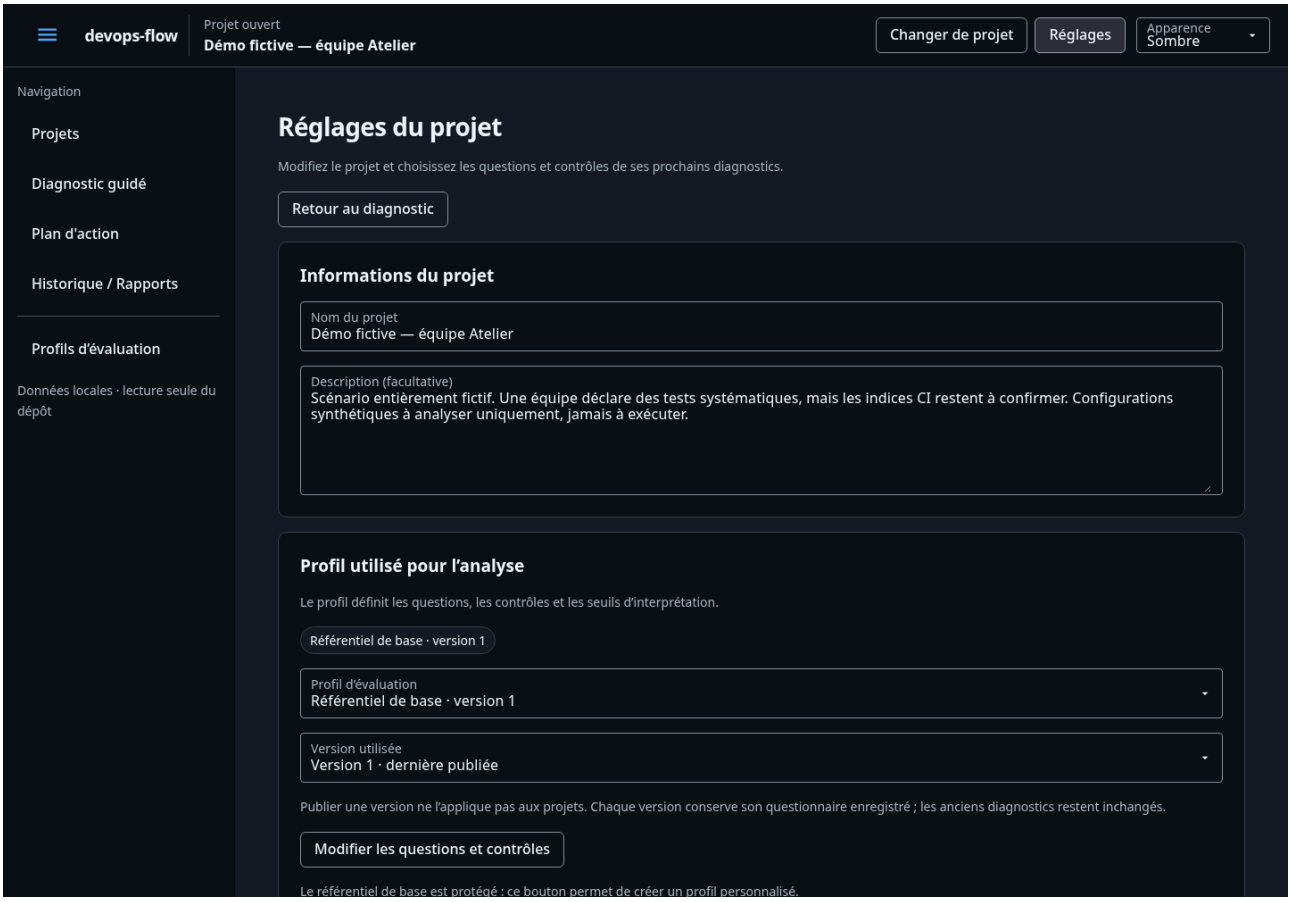
Déclarer ou corriger une stack permet de créer une déclaration pour un chemin relatif ou de corriger une détection. Les champs manuels priment pour leur champ, sans effacer les signaux automatiques. Enregistrer invalide l'analyse courante ; relancer le scan, et sélectionner à nouveau le ZIP si nécessaire. Une déclaration est inactive lorsque son chemin n'est pas retrouvé. Après un conflit entre sessions, utiliser **Recharger les déclarations** avant de réessayer.

Dans **4. Résultats**, l'architecture appartient au diagnostic sélectionné. Les constats se filtrent par composant et leurs actions se suivent indépendamment. Un changement de déclaration ne modifie jamais ce résultat figé. Un ancien diagnostic sans architecture reste consultable sans recalcul. Les fichiers non observés sont des besoins potentiels à vérifier, jamais des obligations établies.

Diagnostic guidé — apparence sombre, données fictives



Réglages du projet et profil utilisé



Projet et diagnostic sur écran mobile

